

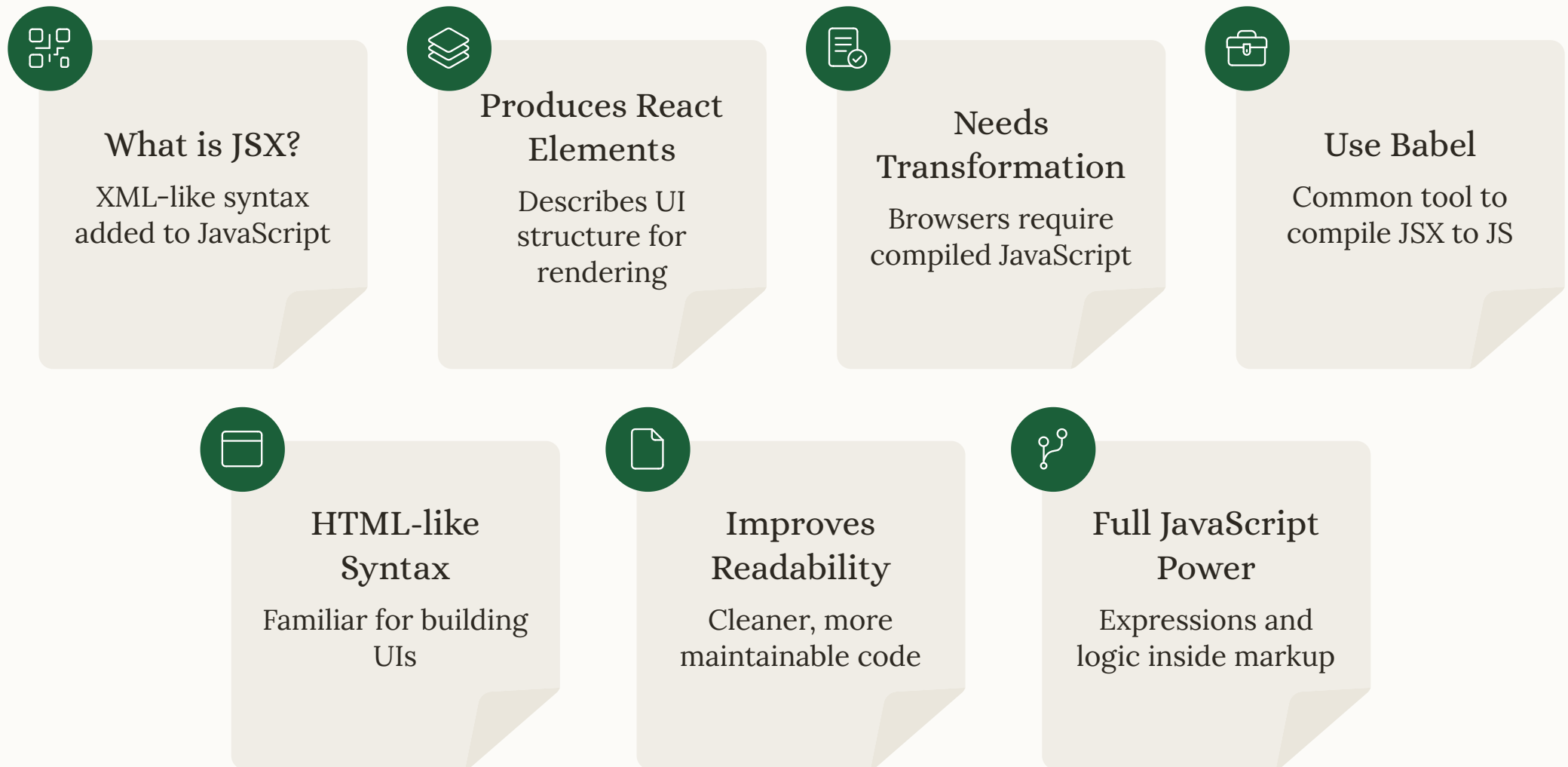


JSX (JavaScript XML) in React

FUNDAMENTALS

Introduction to JSX

JSX, or JavaScript XML, is a syntax extension for JavaScript that allows developers to write HTML-like code within JavaScript files. It is not mandatory for React development, but it has become the standard due to its intuitive nature and readability. JSX makes it easier to visualise the structure of user interfaces whilst maintaining the full power of JavaScript.



- ❑ JSX combines the declarative nature of HTML with the dynamic capabilities of JavaScript, creating a powerful tool for building interactive user interfaces.

SYNTAX

JSX Syntax and Essential Rules

Understanding JSX syntax is crucial for React development. JSX follows specific rules that distinguish it from standard HTML whilst maintaining readability. Mastering these rules ensures that your React components render correctly and your code remains maintainable.



Embedding JavaScript Expressions

Use curly braces {} to embed any valid JavaScript expression within JSX. This allows dynamic content rendering based on variables, function calls, or calculations.



Single Root Element Rule

Every JSX expression must have exactly one parent element. If returning multiple elements, wrap them in a single container like <div> or use React Fragments <>.



Self-Closing Tags

All tags must be properly closed. Tags without children must be self-closing with a forward slash, such as or <input />.

```
// Embedding JavaScript expressions
const name = "Student";
const element = <h1>Hello, {name}!</h1>;
```

```
// Single root element
const component = (
  <div>
    <h1>Title</h1>
    <p>Content</p>
  </div>
);
```

COMPARISON

JSX versus HTML: Key Differences

Whilst JSX resembles HTML, several important differences exist due to JavaScript's reserved keywords and naming conventions. Understanding these distinctions prevents common errors and ensures your React code functions correctly.

Attribute Naming Conventions

JSX uses camelCase for attribute names instead of HTML's kebab-case. For example, `class` becomes `className`, and `for` becomes `htmlFor`. This is necessary because "class" and "for" are reserved words in JavaScript.

Style Attributes

The `style` attribute accepts a JavaScript object with camelCase properties rather than a CSS string. For instance, `background-color` becomes `backgroundColor`, and values must be strings or numbers.

Event Handlers

Event handler names use camelCase in JSX. Write `onClick` instead of `onclick`, and `onChange` instead of `onchange`. Event handlers receive synthetic event objects, not native browser events.

HTML Example

```
<div class="container">
  <label for="email">Email</label>
  <input onclick="handleClick()" />
</div>
```

JSX Example

```
<div className="container">
  <label htmlFor="email">Email</label>
  <input onClick={handleClick} />
</div>
```



COMPONENTS

Using JSX with React Components

JSX integrates seamlessly with React components, enabling developers to create reusable UI elements. Both functional and class components utilise JSX to define their render output, with functional components being the modern standard approach.

01

Functional Components

Modern React favours functional components —JavaScript functions that return JSX. They are concise, easier to test, and work with React Hooks for state management.

02

Class Components

Traditional class components extend `React.Component` and use a `render()` method to return JSX. Whilst still supported, they are less common in new codebases.

03

Passing Data with Props

Components accept inputs called "props" (properties). JSX allows you to pass data to components using attributes, enabling dynamic and reusable component structures.

Functional Component Example

```
function Greeting(props) {
  return (
    <div>
      <h2>Welcome, {props.name}!</h2>
      <p>Course: {props.course}</p>
    </div>
  );
}

// Usage
<Greeting
  name="Student"
  course="B.Tech CSE"
/>
```

Class Component Example

```
class Greeting extends React.Component {
  render() {
    return (
      <div>
        <h2>Welcome, {this.props.name}!</h2>
        <p>Course: {this.props.course}</p>
      </div>
    );
  }
}
```



ANALYSIS

Advantages and Limitations of JSX

JSX has transformed React development by offering a more intuitive way to build user interfaces. However, understanding both its strengths and constraints helps developers make informed architectural decisions and troubleshoot effectively.

Advantages

- **Enhanced Readability:** JSX's HTML-like syntax makes component structures immediately understandable, reducing cognitive load
- **Type Safety:** Many errors are caught during compilation rather than at runtime, improving code reliability
- **Maintainability:** Combining markup and logic in one location simplifies updates and refactoring
- **Developer Experience:** Better IDE support with syntax highlighting, autocomplete, and error detection
- **Component Composition:** Facilitates building complex UIs from smaller, reusable pieces



Limitations

- **Build Step Required:** JSX must be transpiled to JavaScript using tools like Babel, adding complexity to the development pipeline
- **Learning Curve:** Developers must understand both JavaScript and JSX-specific syntax rules
- **Mixing Concerns:** Some argue that combining markup and logic violates separation of concerns principles
- **Debugging Complexity:** Stack traces reference transpiled code, which can complicate error diagnosis

- ❑ Babel is the most common transpiler that converts JSX into `React.createElement()` function calls that browsers can execute.

SUMMARY

Key Takeaways and Learning Outcomes

This presentation has covered the fundamental concepts of JSX and its role in React development. These key points form the foundation for building React applications and are essential knowledge for your academic progression.

Core Concepts

- JSX is a syntax extension that allows HTML-like code in JavaScript
- It requires transpilation by Babel before browser execution
- JSX expressions must have a single root element
- JavaScript expressions are embedded using curly braces {}

Practical Application

- Use camelCase for attributes (className, onClick)
- JSX works with both functional and class components
- Props enable data passing between components
- Understand differences between JSX and HTML syntax

Examination Focus

- Explain JSX syntax rules and single root element requirement
- Identify differences between JSX and HTML attributes
- Demonstrate embedding JavaScript expressions in JSX
- Discuss advantages of JSX in React development
- Understand the role of Babel in JSX transpilation

"JSX is not a requirement for using React, but it makes writing React applications significantly more intuitive and maintainable. Mastering JSX is essential for modern front-end development."

